EXAMINATION RESULT SYSTEM

Python Programming Case Study

1. Problem Statement

Calculating examination results for multiple subjects involves adding marks, applying grace marks where permitted, identifying failed subjects, and assigning an overall class. Performing these steps manually can cause errors. A Python program can automate the calculations and display the student's total, percentage, and final result classification.

2. Objective

To develop a Python program that accepts marks for a specified number of subjects, applies grace marks to eligible scores, calculates total marks and average percentage, checks for failed subjects, and displays the final result and class.

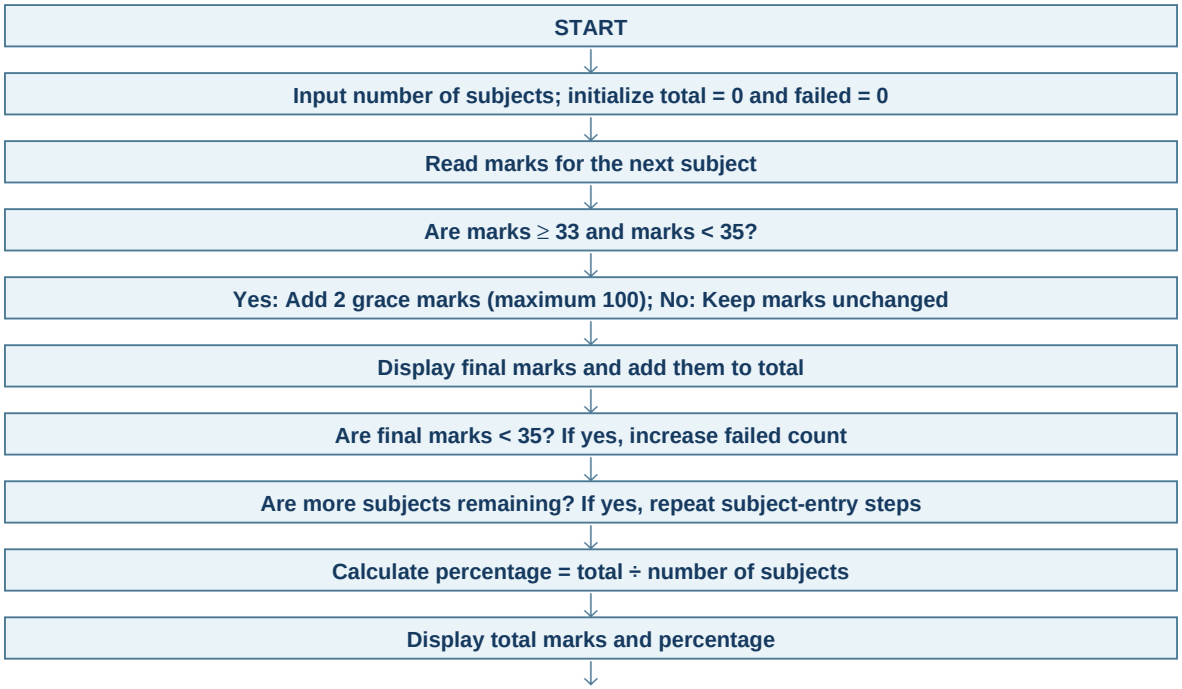
3. Assumptions

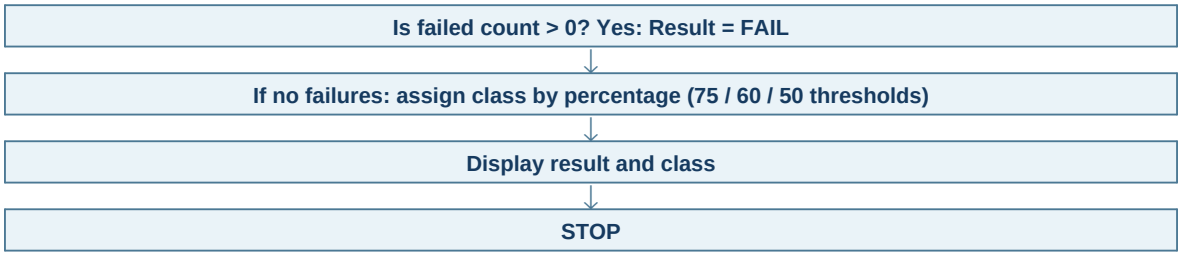
• The number of subjects is a positive integer.
• Marks for each subject are entered as numeric values between 0 and 100.
• If a subject mark is at least 33 but below 35, 2 grace marks are added, up to a maximum of 100.
• A final subject mark below 35 is treated as a failure.
• The percentage is calculated as total final marks divided by the number of subjects; each subject is out of 100.
• If one or more subjects are failed, the overall result is FAIL, regardless of the percentage.
• If no subject is failed, class is based on percentage: 75% or above—Distinction; 60% or above—First Class; 50% or above—Second Class; otherwise—Third Class.

4. Algorithm

1. Start the program and read the number of subjects.
2. Initialize total marks and failed-subject count to zero.
3. For each subject, read the marks.
4. If marks are at least 33 and below 35, add 2 grace marks, without exceeding 100.
5. Display the final marks and add them to the total.
6. If final marks are below 35, increase the failed-subject count.
7. After all subjects, calculate percentage as total marks divided by number of subjects.
8. Display total marks and percentage.
9. If any subject is failed, display FAIL. Otherwise, assign the class using the percentage thresholds.
10. Stop the program.

5. Flowchart



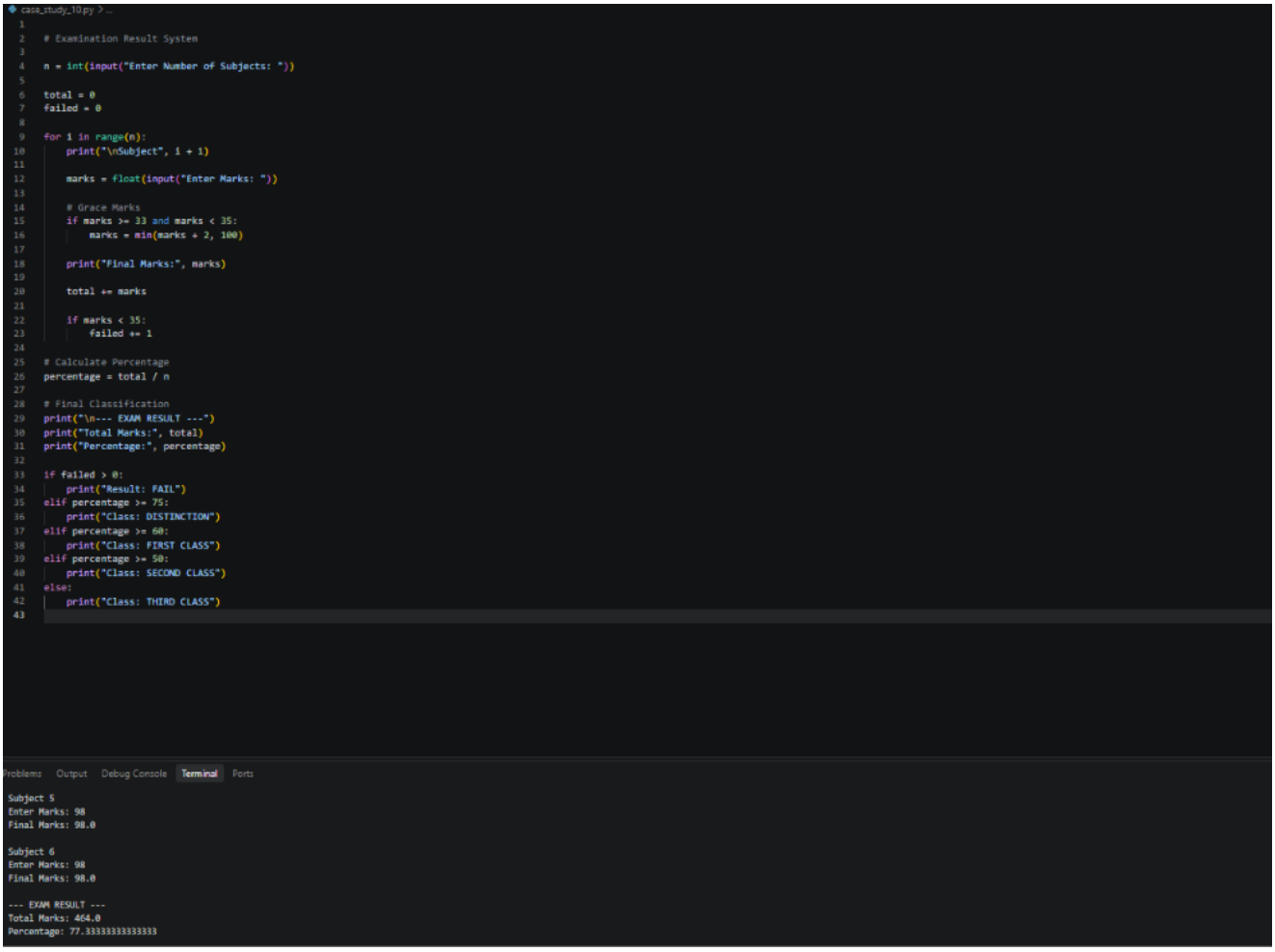

Is failed count > 0? Yes: Result = FAIL
If no failures: assign class by percentage (75 / 60 / 50 thresholds)
Display result and class
STOP

6. Pseudocode

```
BEGIN
INPUT number_of_subjects
total = 0; failed = 0
FOR each subject
INPUT marks
IF marks >= 33 AND marks < 35 THEN
marks = MIN(marks + 2, 100)
END IF
DISPLAY final marks
total = total + marks
IF marks < 35 THEN failed = failed + 1
END FOR
percentage = total / number_of_subjects
DISPLAY total, percentage
IF failed > 0 THEN DISPLAY 'FAIL'
ELSE IF percentage >= 75 THEN DISPLAY 'DISTINCTION'
ELSE IF percentage >= 60 THEN DISPLAY 'FIRST CLASS'
ELSE IF percentage >= 50 THEN DISPLAY 'SECOND CLASS'
ELSE DISPLAY 'THIRD CLASS'
END IF
END
```

7. Python Source Code and Output Screenshot

The original screenshot supplied for this case study is included below.



```
case_study_10.py > ...
1
2   # Examination Result System
3
4   n = int(input("Enter Number of Subjects: "))
5
6   total = 0
7   failed = 0
8
9   for i in range(n):
10      print("\nSubject", i + 1)
11
12      marks = float(input("Enter Marks: "))
13
14      # Grace Marks
15      if marks >= 33 and marks < 35:
16          marks = min(marks + 2, 100)
17
18      print("Final Marks:", marks)
19
20      total += marks
21
22      if marks < 35:
23          failed += 1
24
25  # Calculate Percentage
26  percentage = total / n
27
28  # Final Classification
29  print("\n--- EXAM RESULT ---")
30  print("Total Marks:", total)
31  print("Percentage:", percentage)
32
33  if failed > 0:
34      print("Result: FAIL")
35  elif percentage >= 75:
36      print("Class: DISTINCTION")
37  elif percentage >= 60:
38      print("Class: FIRST CLASS")
39  elif percentage >= 50:
40      print("Class: SECOND CLASS")
41  else:
42      print("Class: THIRD CLASS")
43
```

```
Problems  Output  Debug Console  Terminal  Ports

Subject 5
Enter Marks: 98
Final Marks: 98.0

Subject 6
Enter Marks: 98
Final Marks: 98.0

--- EXAM RESULT ---
Total Marks: 464.0
Percentage: 77.33333333333333
```

8. Conclusion

The Examination Result System automates subject-wise grace-mark adjustments, total and percentage calculation, failure checking, and class assignment. It demonstrates the practical use of Python loops, conditional statements,

arithmetic operations, and the min() function to produce a consistent examination report.